Singly Linked List – Delete From Beginning And Time Complexity.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}

void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
```

```
        return;
    }

    newNode->data = data;

    if (position == 1)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}

void insertAtPosition2(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (position == 0)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 0)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 0;
```

```
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }

        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}

void deleteFromBeginning(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *temp = *head;
        *head = (*head)->next;
        cout << "Deleted node: " << temp->data << endl;
        temp->next = nullptr;
        free(temp);
        temp = nullptr;
    }
    cout << "Node is deleted successfully." << endl;
}
```

```
int main()
{
   createEmptyList(&head);
    while (1)
    {

        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;
        cout << "3. Insert at a position,indexing node from 1 to N" << endl;
        cout << "4. Insert at a position,indexing node from 0 to N-1" << endl;
        cout << "5. Delete from the beginning" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
      case 2:
      {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtEnd(&head, data);
            break;
      }
     case 3:
        {
            int data,position;
```

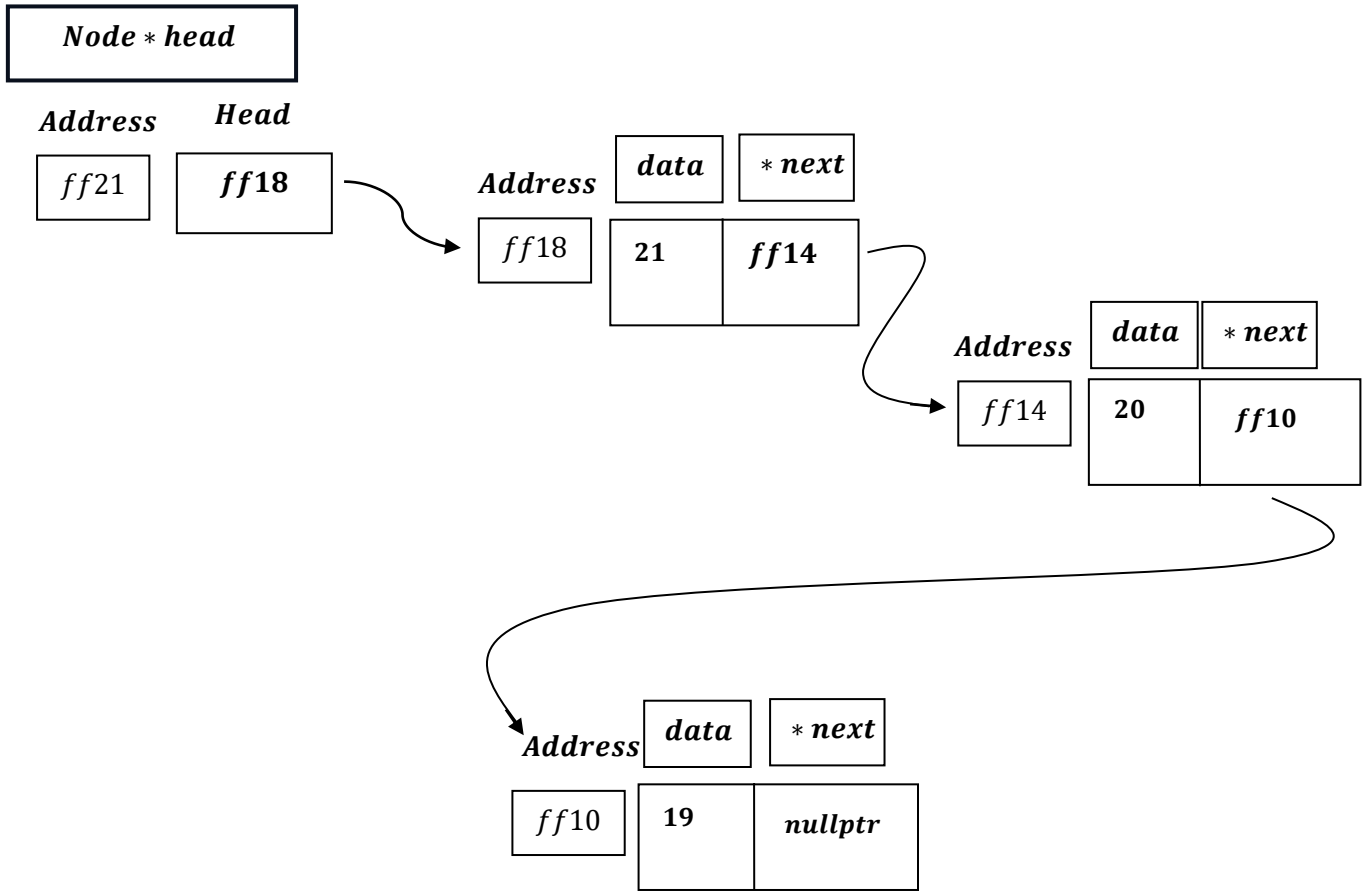
```
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
           }

        case 4:
        {
            cout << "Insert at a position,indexing node from 0 to N-1:" << endl;
            int data, position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition2(&head, data, position);

            break;
        }
        case 5:
        {
            deleteFromBeginning(&head);
            break;
        }
       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Program Analysis:

We recall previous formed linked list:



At first we move to the function call:

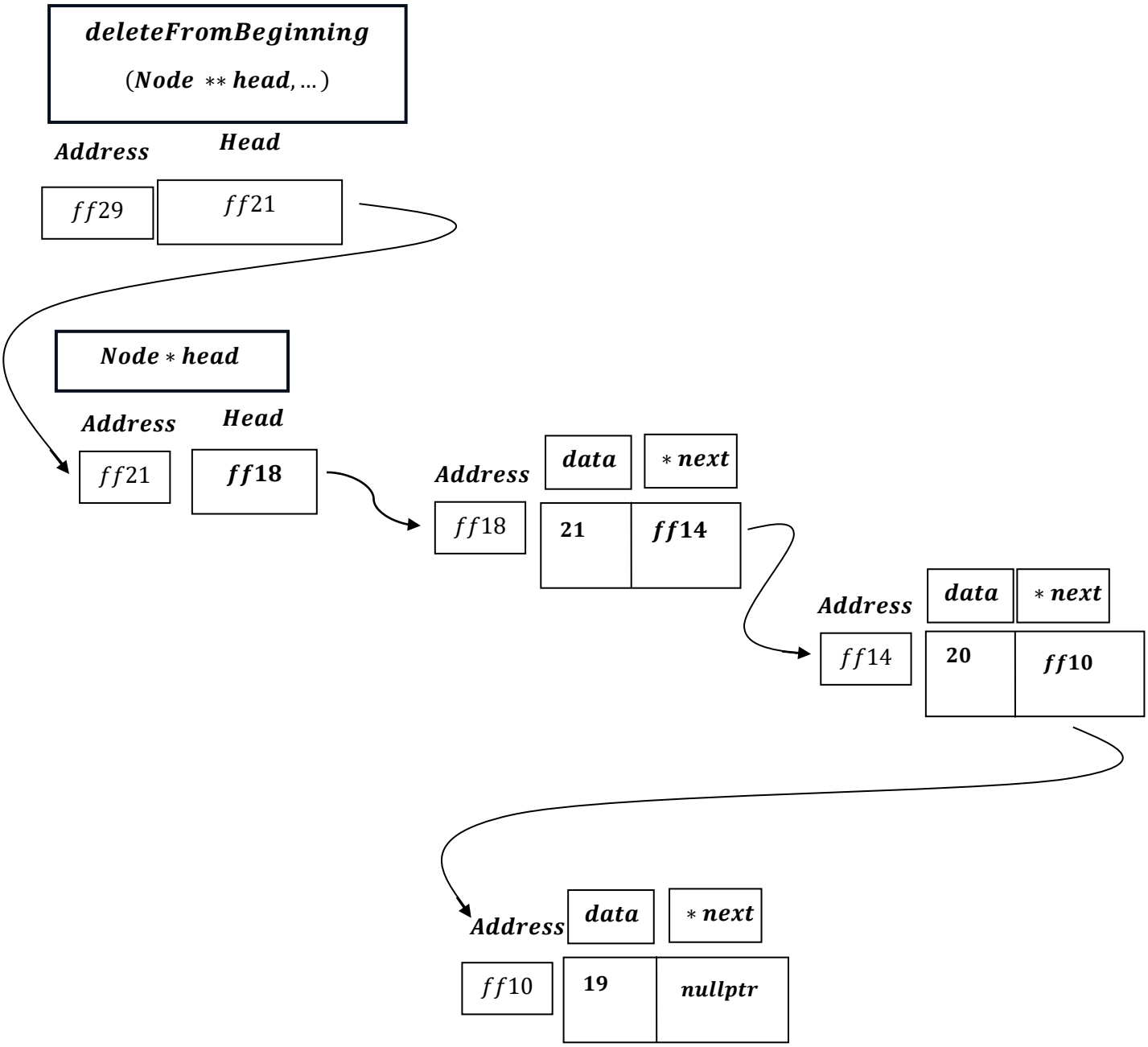
```
deleteFromBeginning(&head);
```

Hence, at function call we pass:

$$deleteFromBeginning(ff18); \rightarrow \textit{That is the address of the `head`}.$$

Next :

```
void deleteFromBeginning(Node **head) {…}
```

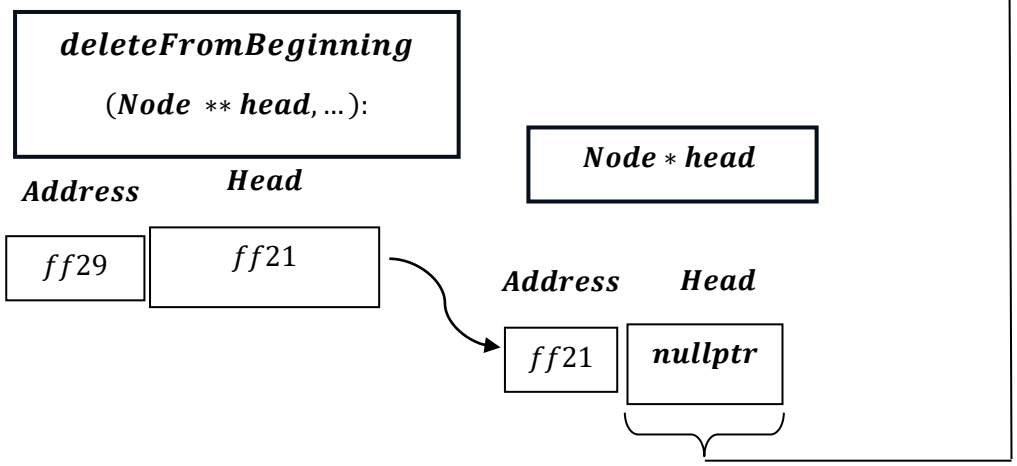


Now if Head pointer of Node structure points to nullptr then list is empty , i.e. Head carries value nullptr (not pointing to any valid memory location).

```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```

$if(*head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the deleteFromBeginning function.



Then , it prints a message : `Error: List is empty.`

Next is else part:

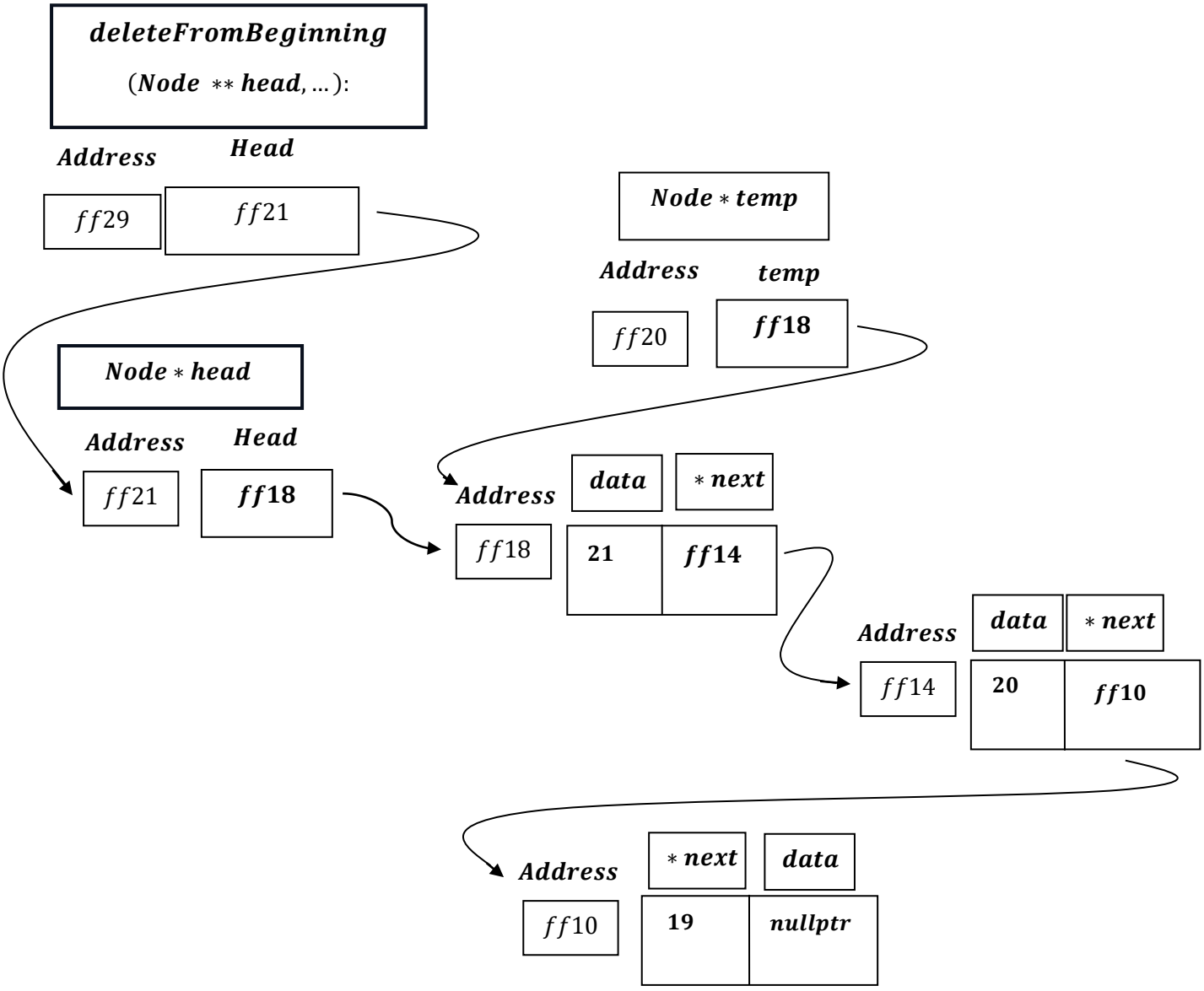
```
    else
    {
        Node *temp = *head;
        *head = (*head)->next;
        cout << "Deleted node: " << temp->data << endl;
        temp->next = nullptr;
        free(temp);
        temp = nullptr;
    }
    cout << "Node is deleted successfully." << endl;
}
```

At first:

```
        Node *temp = *head;
```

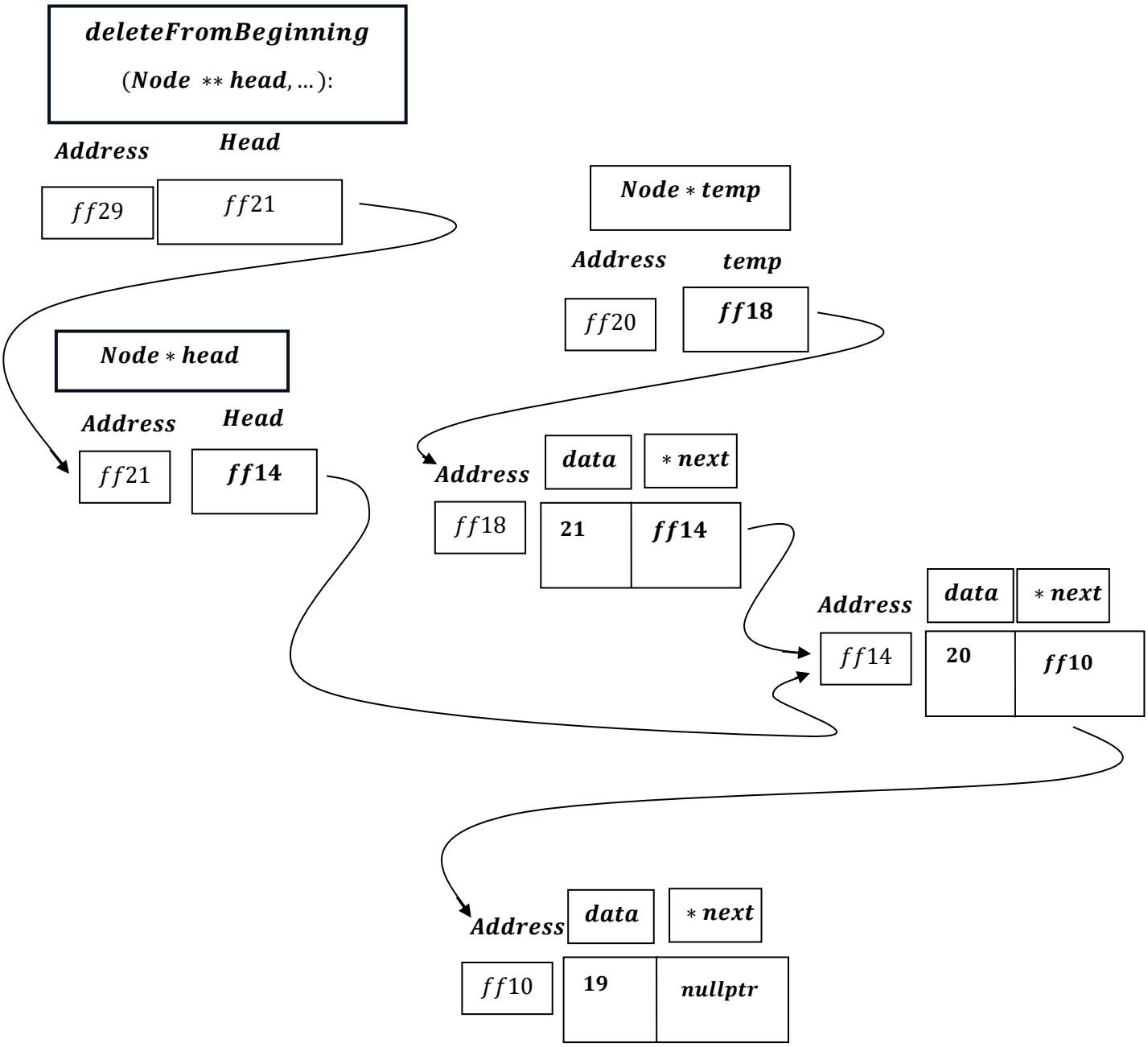
We create a temporary pointer of Node structure i.e.`temp` which carries the value of head pointer of Node structure.

Why ff18 ? ,when we are using * head, i.e. here head used is of local pointer to pointer i.e. Node ** head , and as * head is used, it will first derefer from Node ** head and refer us to Node * head , which will cause current pointer to store value contained in Node * head i.e. ff18 , address of Head node to which Node * head pointer points to.



```
        *head = (*head)->next;
```

Now , head pointer will get assigned to value of its pointed memory address's [containing data and next pointer] next pointer's value.

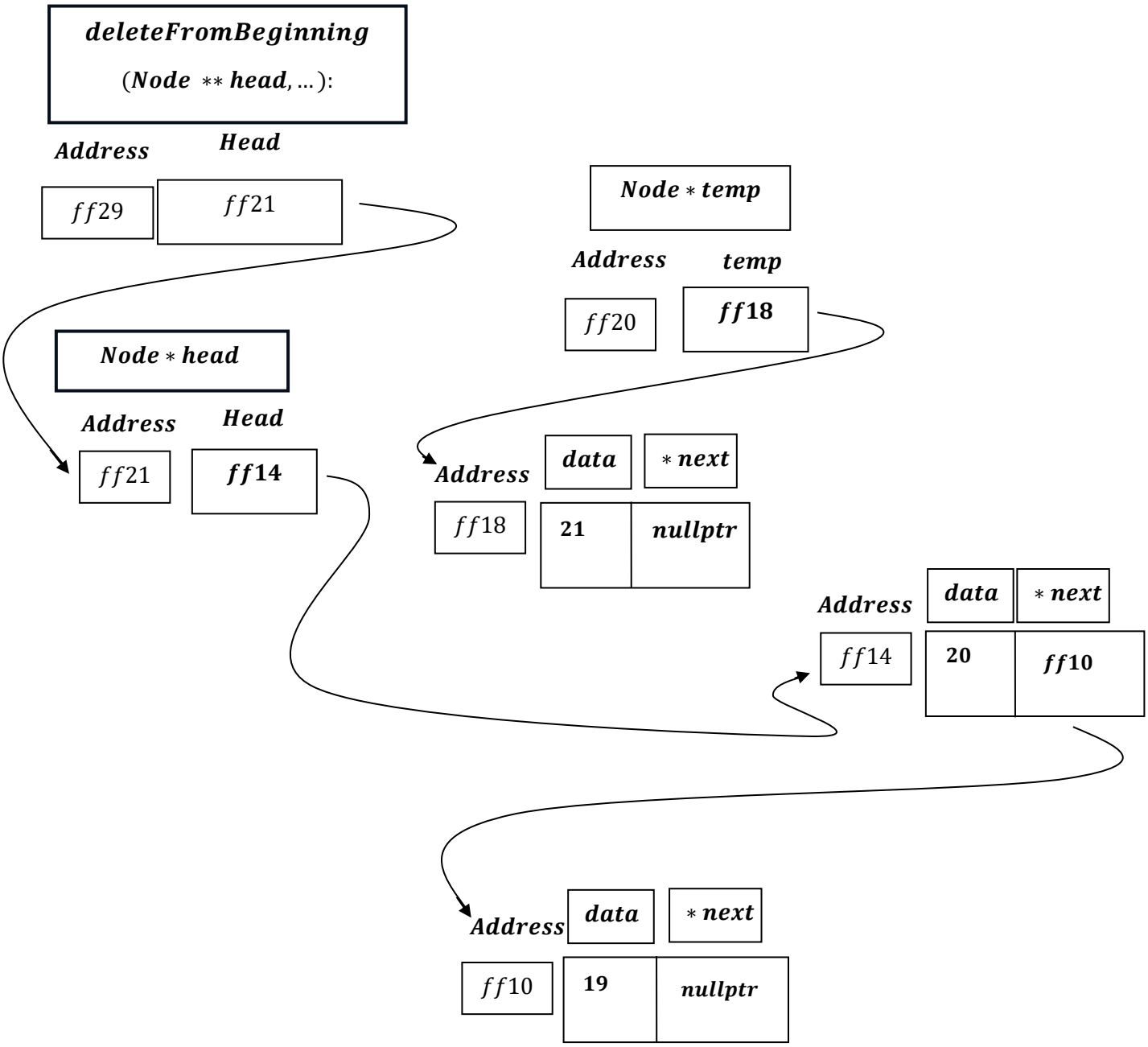



```
        cout << "Deleted node: " << temp->data << endl;
```

Next we print the data to be deleted i.e. here 21.

```
temp->next = nullptr;
```

Now we set temp's pointed memory location's next pointer value to nullptr for isolation.

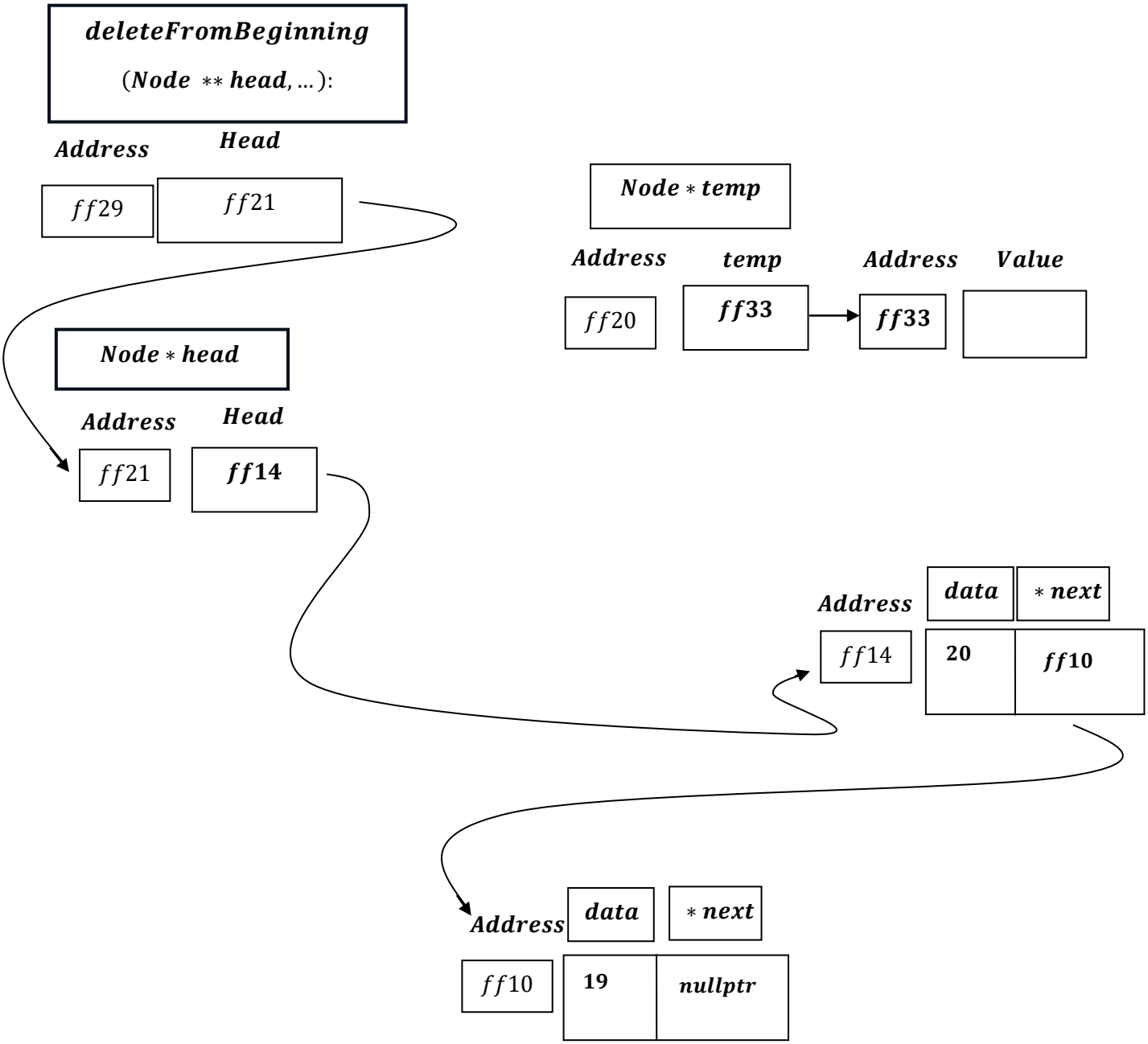


And we free now the memory location pointed by temp pointer through:

```
free(temp);
```

As soon as we free the memory pointed to by temp, both the data and the next pointer associated with that node are deallocated. The free(temp) call releases the memory, making it available for reuse, but it also means that the entire node (including data and next pointer) is no longer accessible.

Now we are left with :



After freeing the memory location , temp valiable is still present pointing

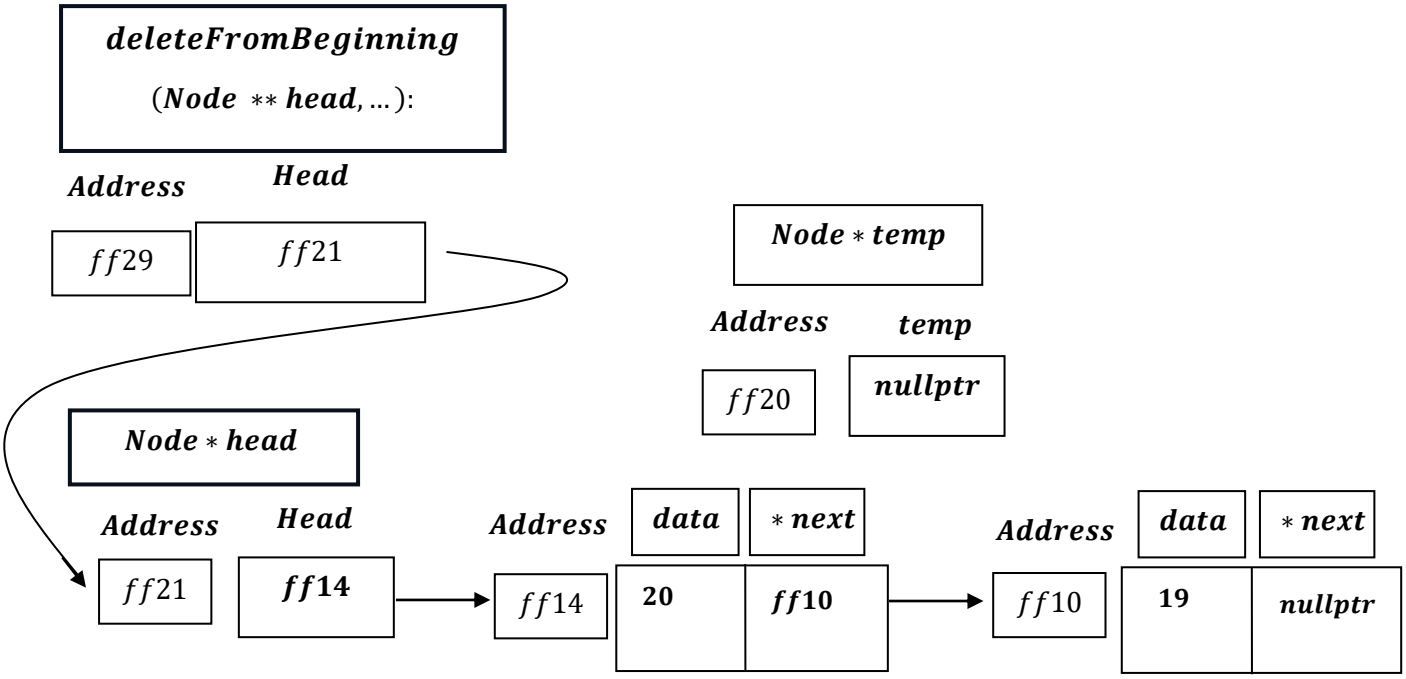
invalid memory location i.e. now temp is a dangling pointer.

A dangling pointer in programming refers to a pointer that continues to point to a memory location after the memory has been deallocated (freed).

Now to avoid dangling pointer:

```
temp = nullptr;
```

We set value of temp pointer to nullptr.

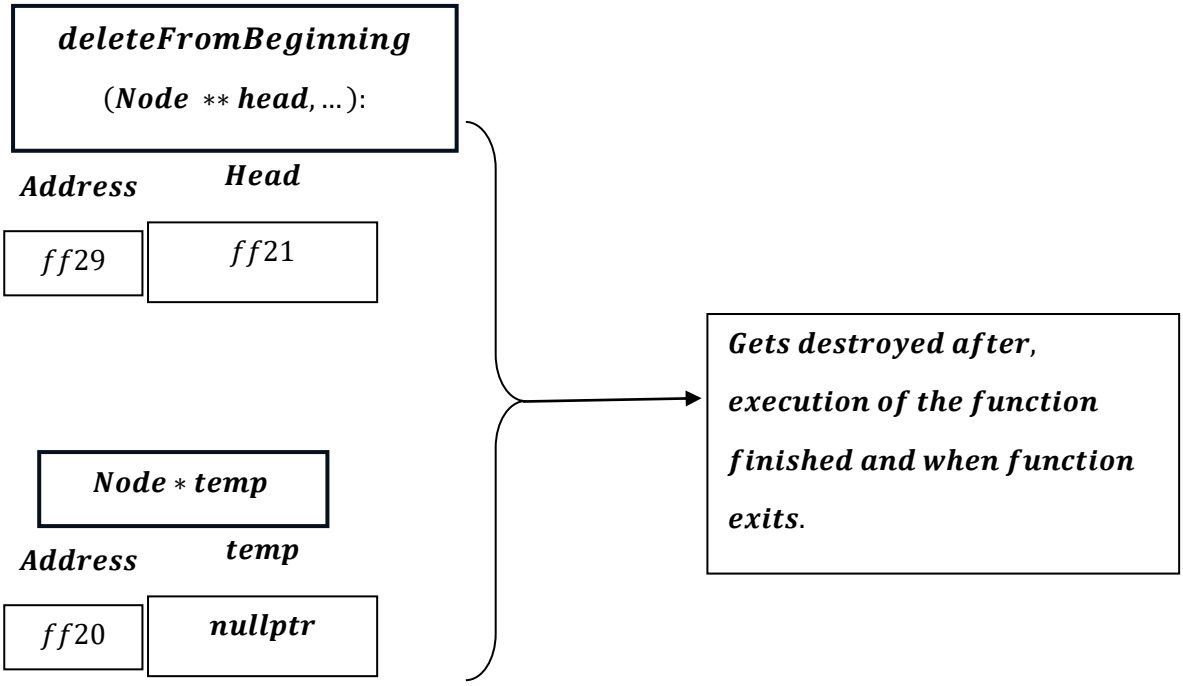


Now we just executes a message that , the `node` deleted successfully:

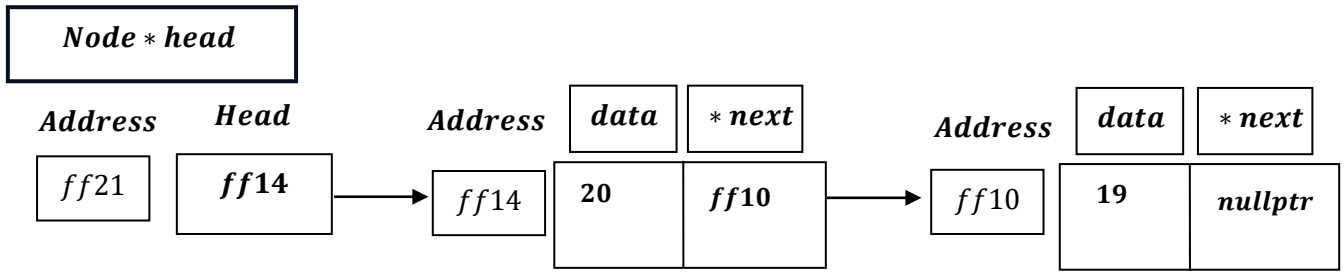
```
    cout << "Node is deleted successfully." << endl;
```

After the function module ends , all temporary pointer variables created gets destroyed.

i.e.



Hence we are left with the list:



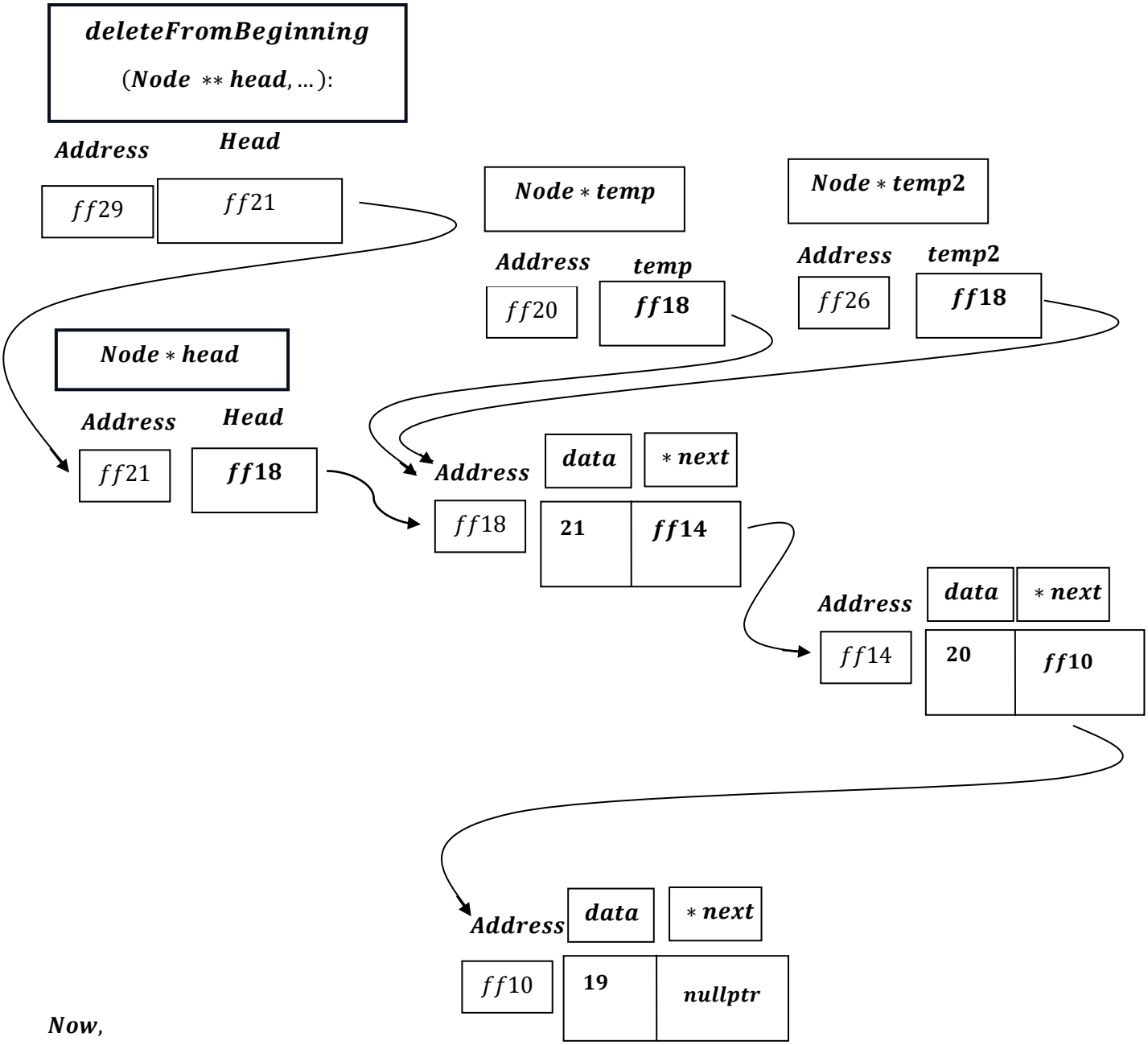
Now , one may question , Why $* \boldsymbol{head} = (* \boldsymbol{head}) \rightarrow \boldsymbol{next}$?

We are going to understand through example:

***Example* 1**:

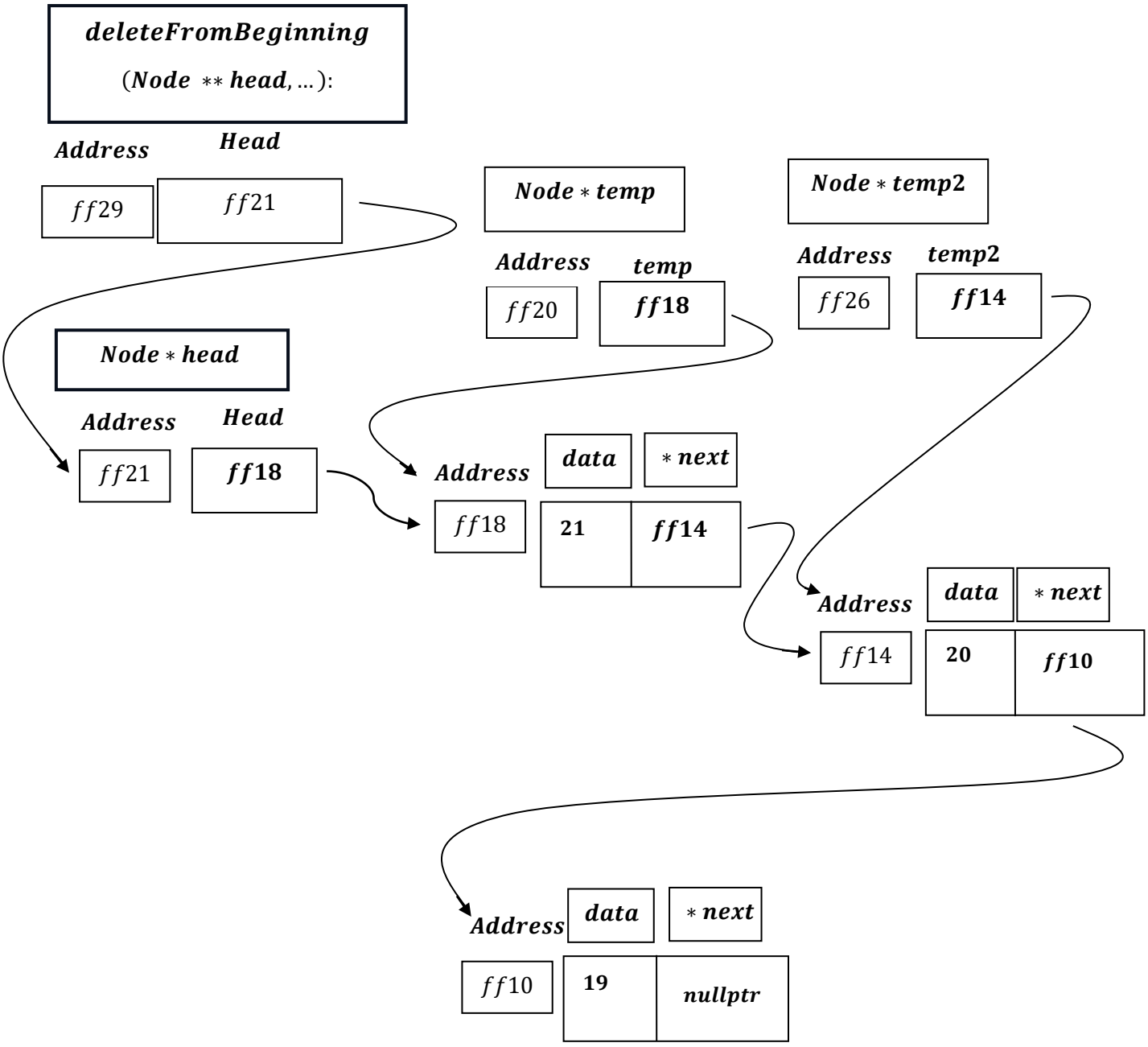
This thing we can also do it through a temporary pointer variable.

```
Node *temp2 = *head;
```



Now,

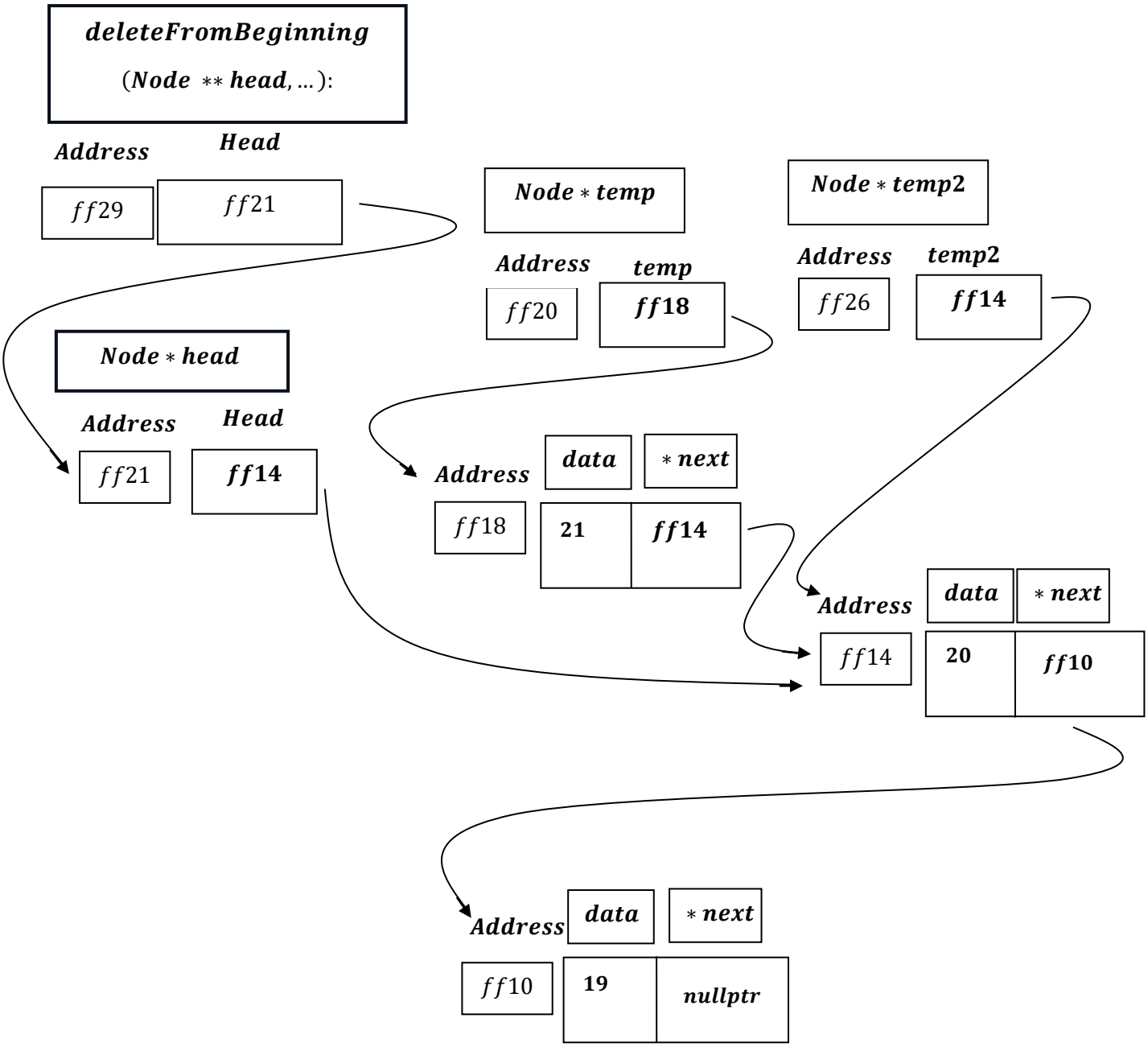
```
temp2 = temp2->next;
```



*As, we can see its a single pointer * temp2 and no other pointer to pointer is pointed to * temp2 , hence temp2 directly access next pointer without braces .*

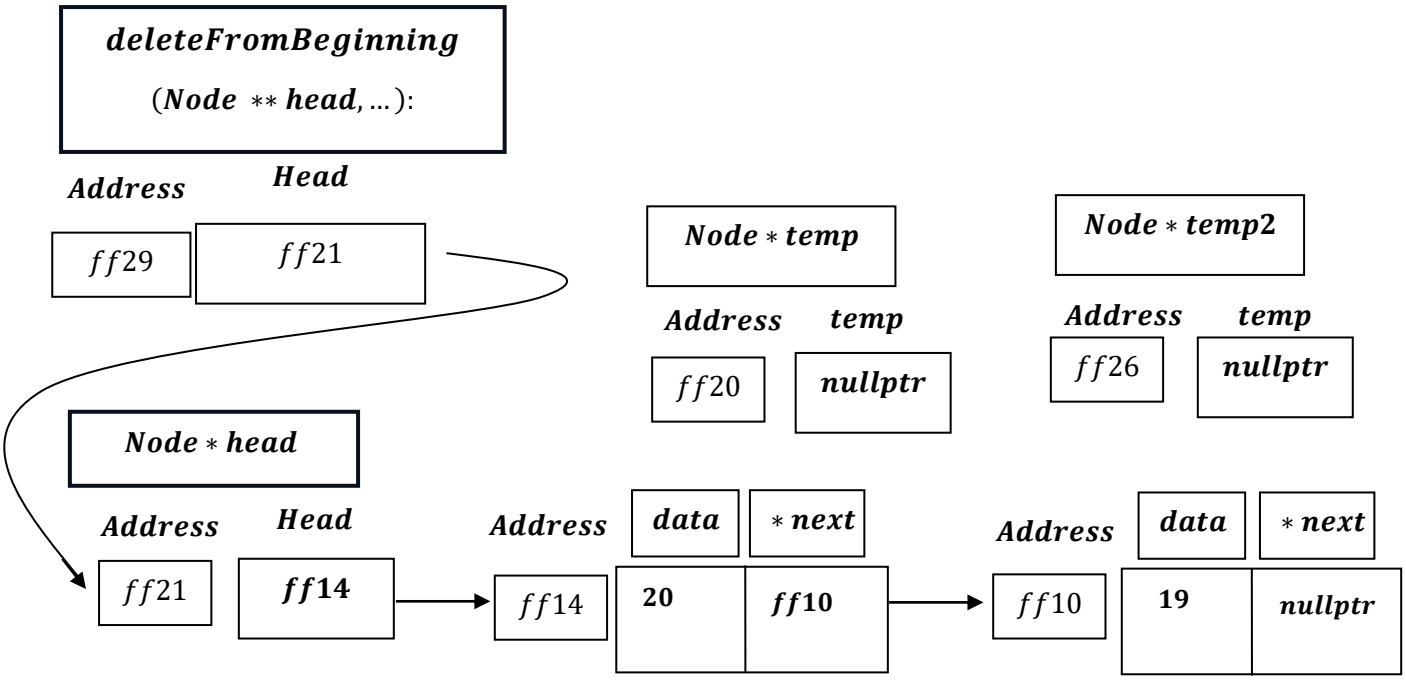
Next:

```
*head = temp2;
```



Now ,we cannot free (temp2) as temp2's allocated memory will be gone, hence after the operation assign nullptr.

```
temp = nullptr;
```

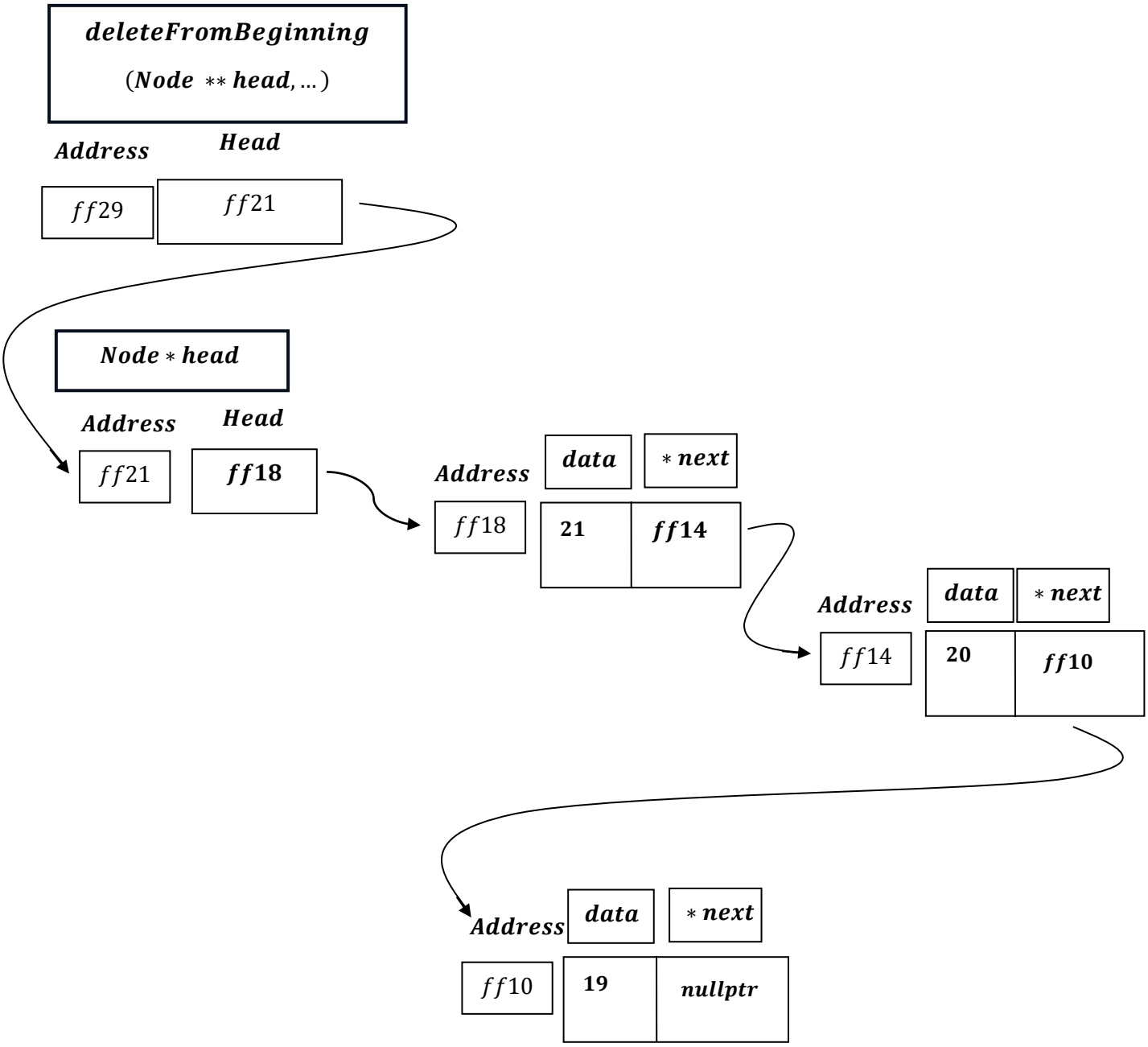


The line of code is given as follows:

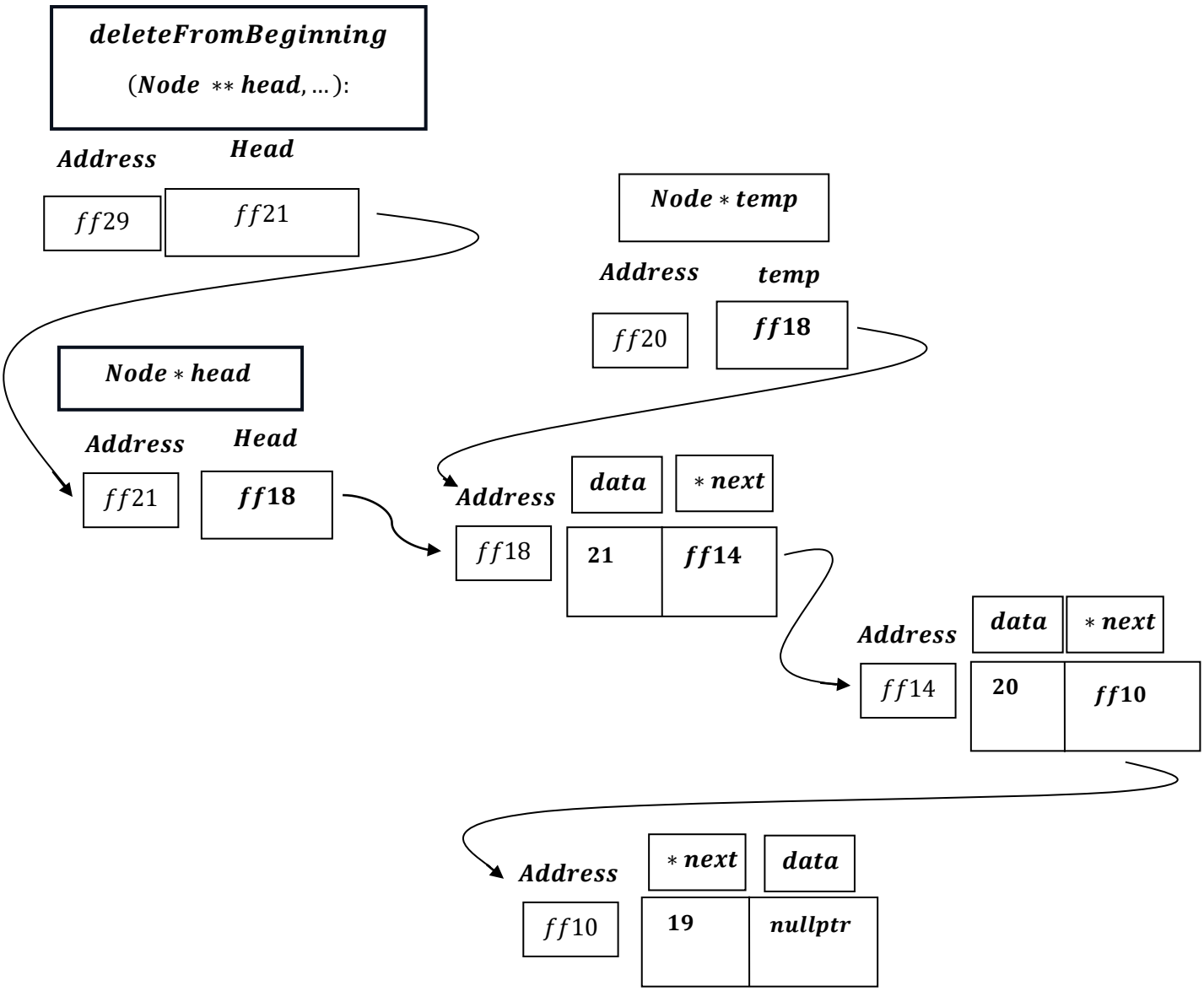
```
void deleteFromBeginning(Node **head) //->6
{
    // Check if the list is empty
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        // Store the head node
        Node *temp = *head;
        Node *temp2 = *head;
        temp2 = temp2->next;
        *head = temp2;
        cout << "Deleted node: " << temp->data << endl;
        temp->next = nullptr;
        temp2 = nullptr;
        free(temp);
```

```
        temp = nullptr;
    }
    cout << "Node is deleted successfully." << endl;
}
```

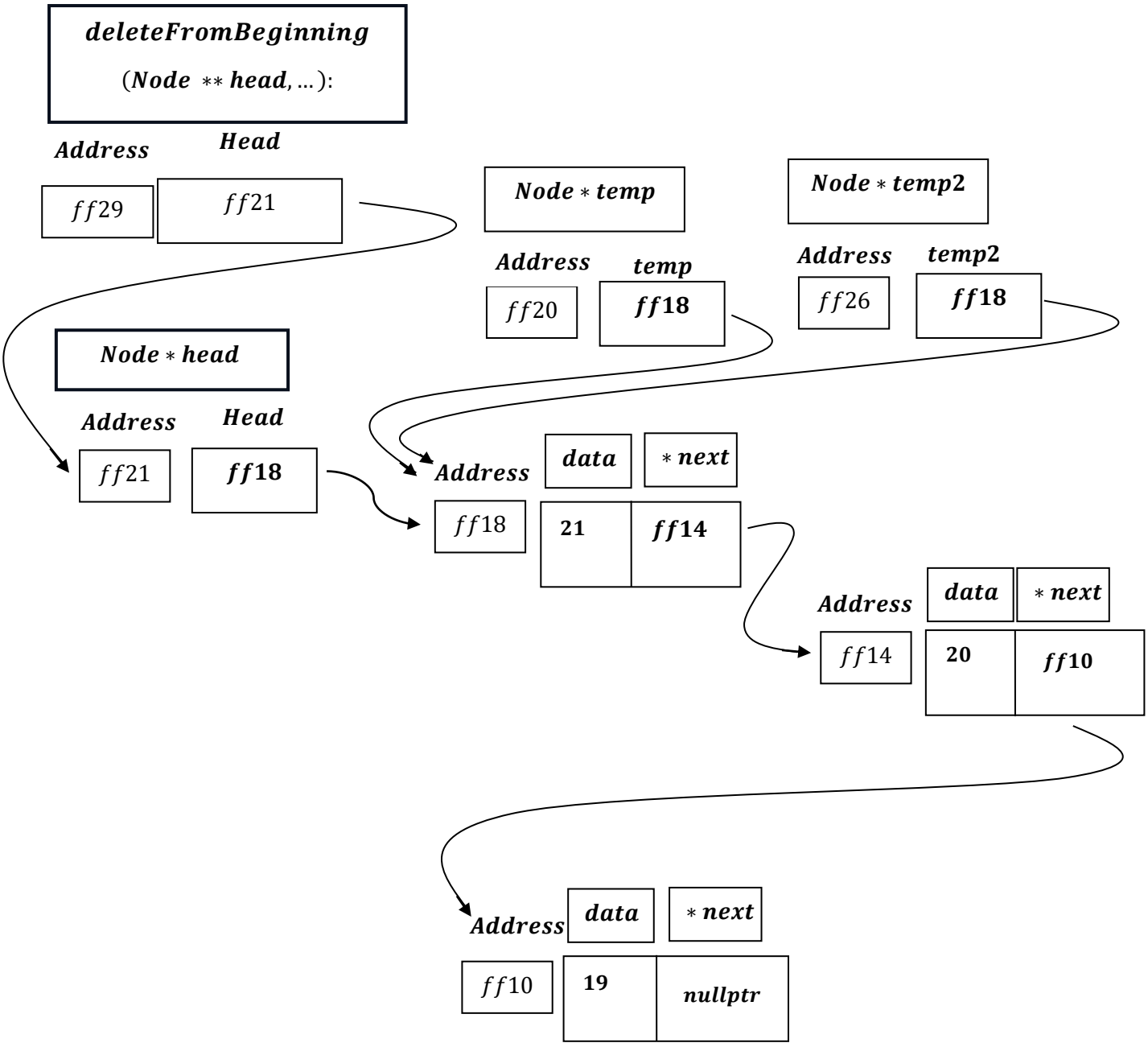
Example 2:



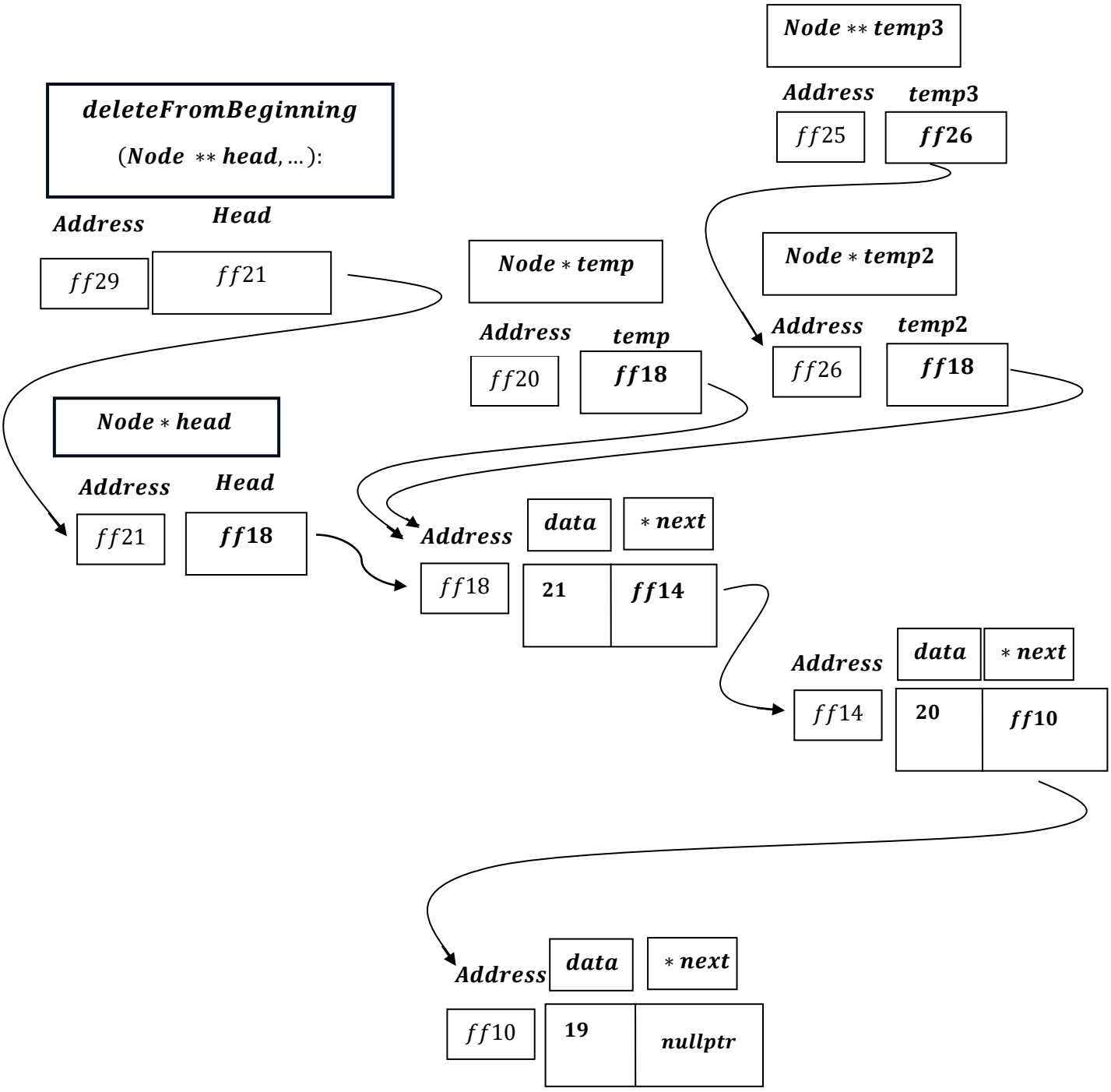
```
Node *temp = *head;
```




```
Node *temp2 = *head;
```




```
Node **temp3= &temp2;
```




```
*temp3 = (*temp3)->next;
```

→ *Here temp3 is of local variable Node ** temp3 and * temp3 derefer from Node ** temp3 and refer us to Node * temp2.*

→ (* temp3), *here temp3 is of local variable Node ** temp3 and*

$*temp3\ derefer\ from\ Node\ **temp3\ and\ refer\ us\ to\ Node\ *temp2.$

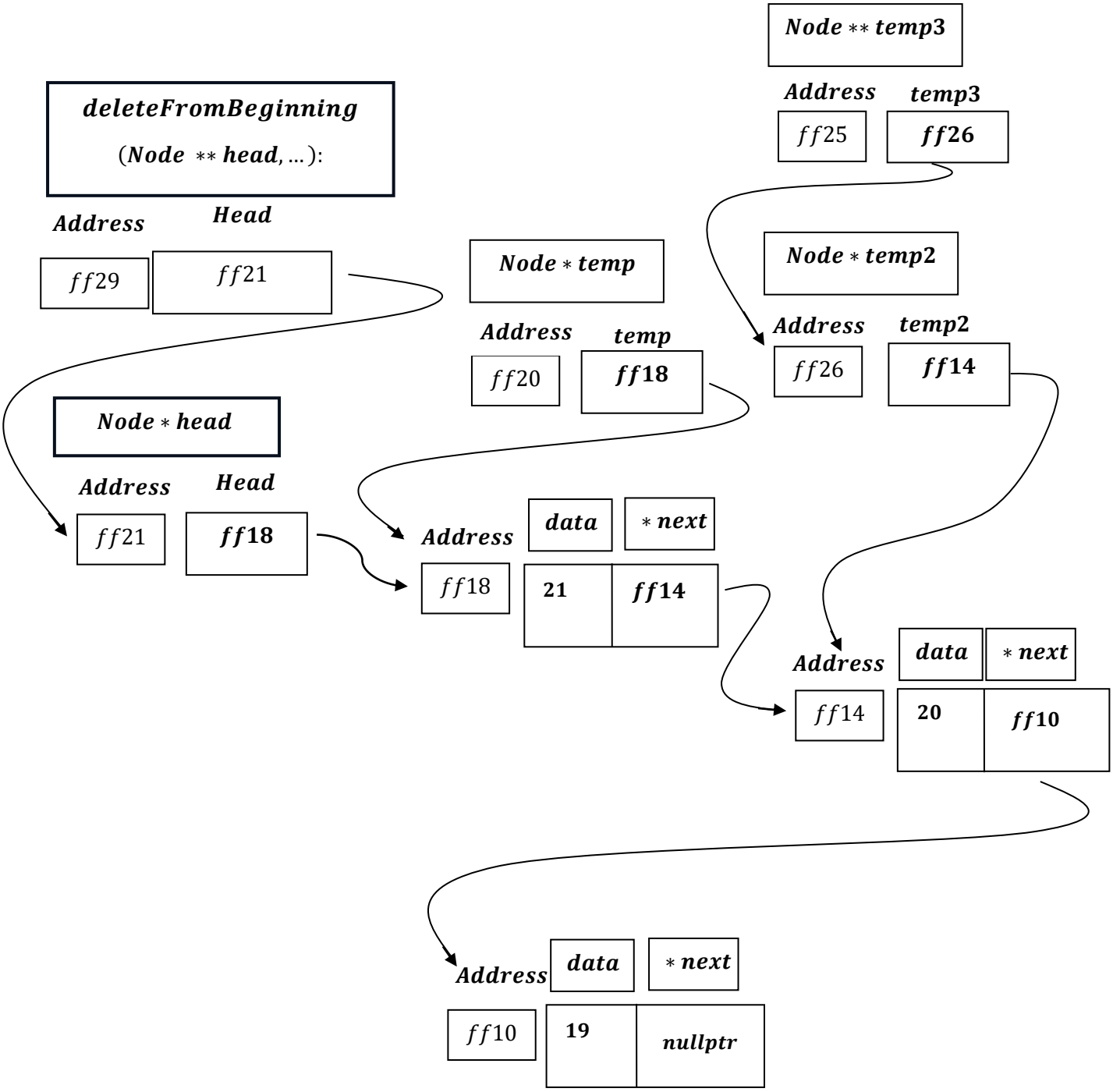
$If\ we\ write\ *temp3 \rightarrow next\ without\ bracket, it\ would\ be\ interpreted\ as$
$*(temp3 \rightarrow next), which\ means\ user\ is\ trying\ to\ access\ the\ next\ member$
$of\ temp3\ itself\ (which\ is\ incorrect\ because\ temp3\ is\ a\ Node\ **type, not\ a$
$Node\ *type).\ Also\ temp3\ doesnot\ point\ to\ any\ next\ pointer\ but\ temp2\ does$
$and\ temp3\ is\ pointer\ to\ pointer\ of\ Node\ structure\ , that\ carries\ address$
$of\ Node\ structure\ pointer\ temp2\ which\ means\ temp3\ points\ to\ temp2\ and$
$temp2\ points\ to\ the\ address\ of\ memory\ which\ contains\ `next`\ pointer.$

$Hence\ , (*temp3) \rightarrow next\ ensures\ that\ temp3\ is\ first\ dereferenced\ to\ get$
$temp2, and\ then\ the\ next\ member\ of\ the\ node\ pointed\ to\ by\ temp2\ is\ accessed.$

$`As\ *temp3 \rightarrow next\ is\ not\ a\ correct\ syntax\ hence\ bracket\ is\ used\ to\ prioritize$
$the\ dereferencing\ of\ Node\ **temp3\ to\ Node\ *temp2\ `$

$That\ is\ the\ same\ reason\ why\ *head = (*head) \rightarrow next\ is\ used\ for.$

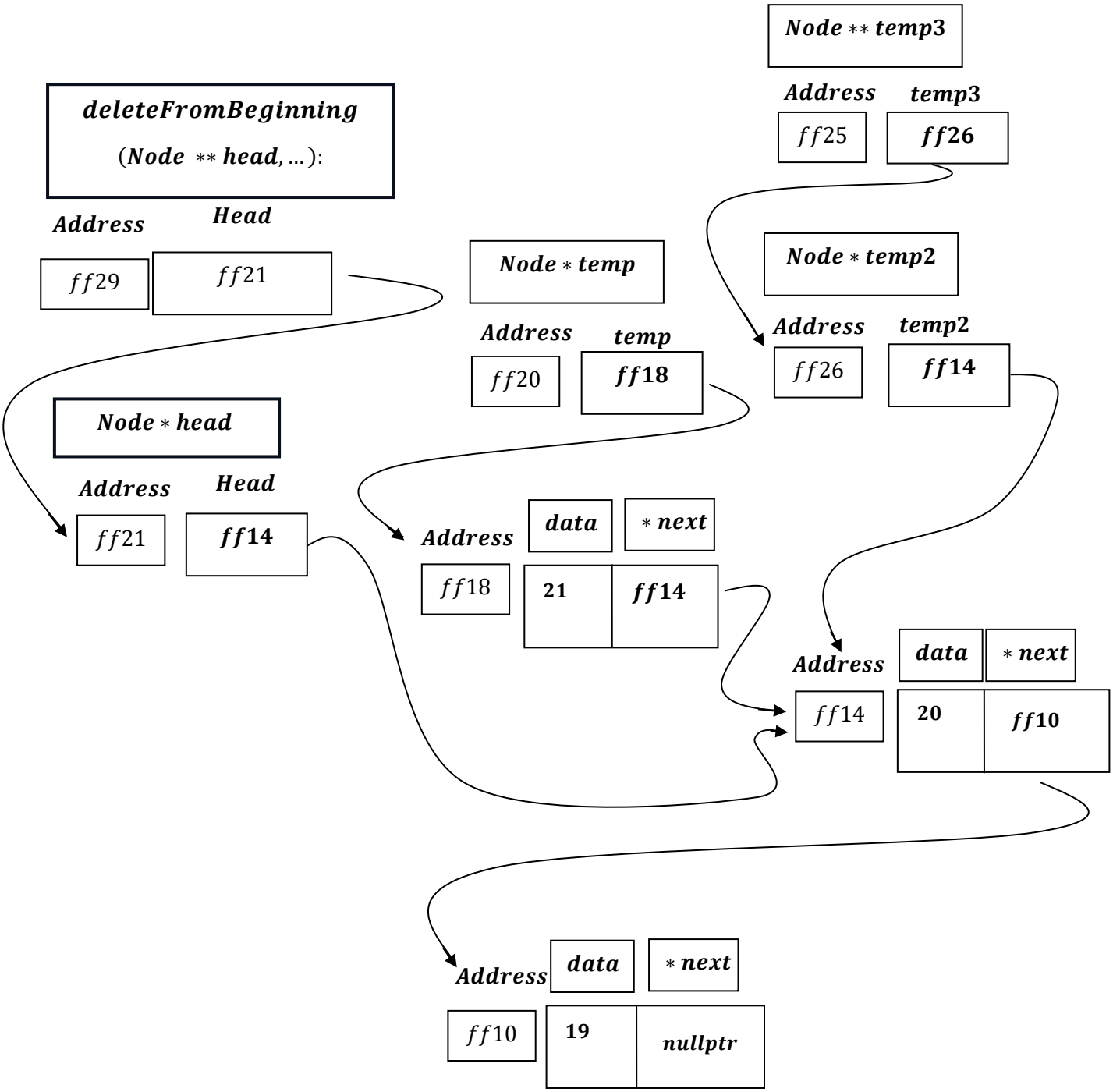
$Now, we\ depict\ this\ pictorally\ as:$


Node ** temp3
Address
temp3
ff25
ff26
deleteFromBeginning
(Node ** head, ...):
Address
Head
ff29
ff21
Node * temp
Node * temp2
Address
temp
ff20
ff18
Address
temp2
ff26
ff14
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr

```
*head = *temp3;
```

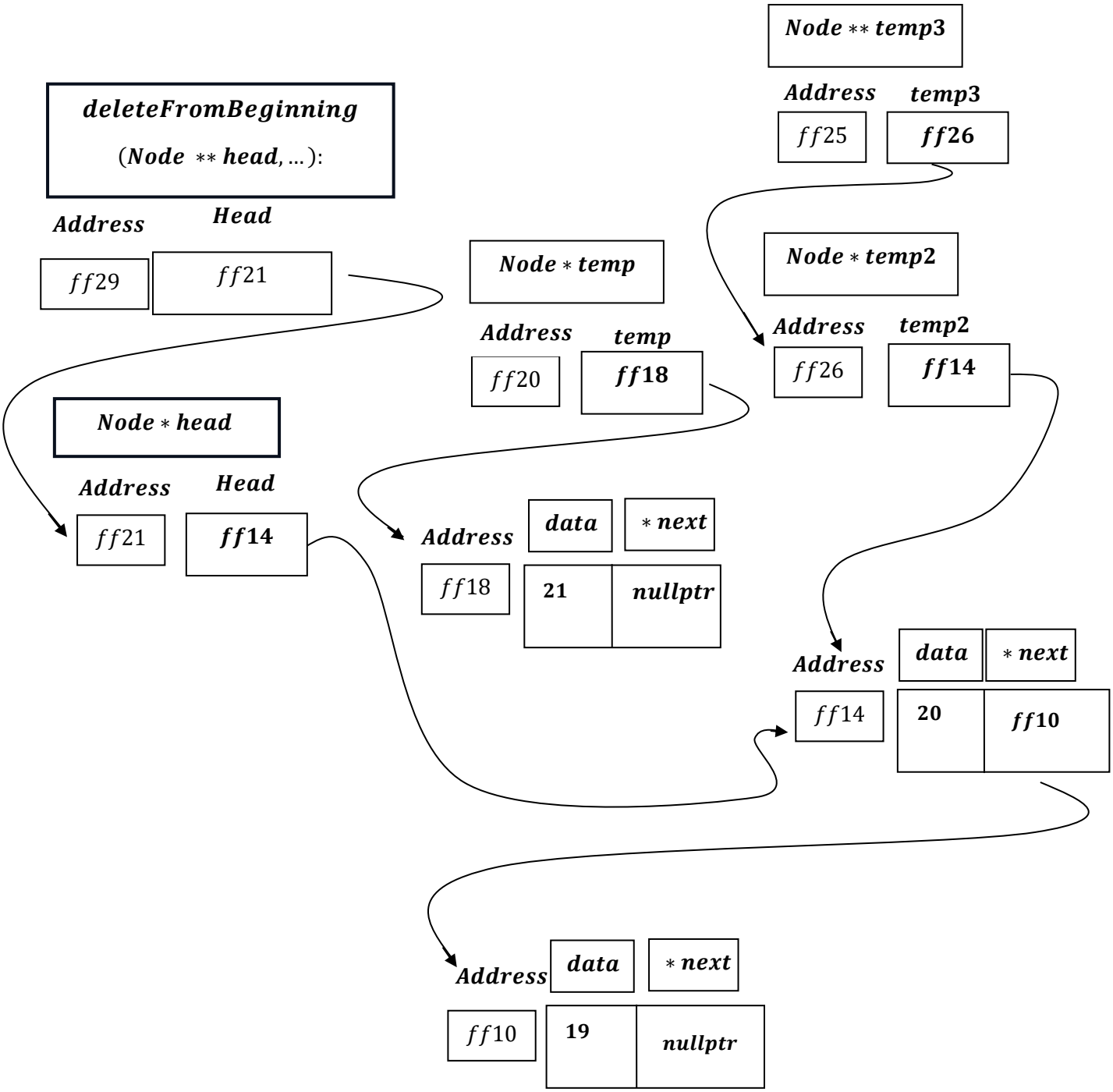
→ *Now, at first Node ** head gets derference to Node * head .*

→ *Then, Node * head gets assigned to Node * temp2 , as Node ** temp3 gets derference to Node * temp2 .*



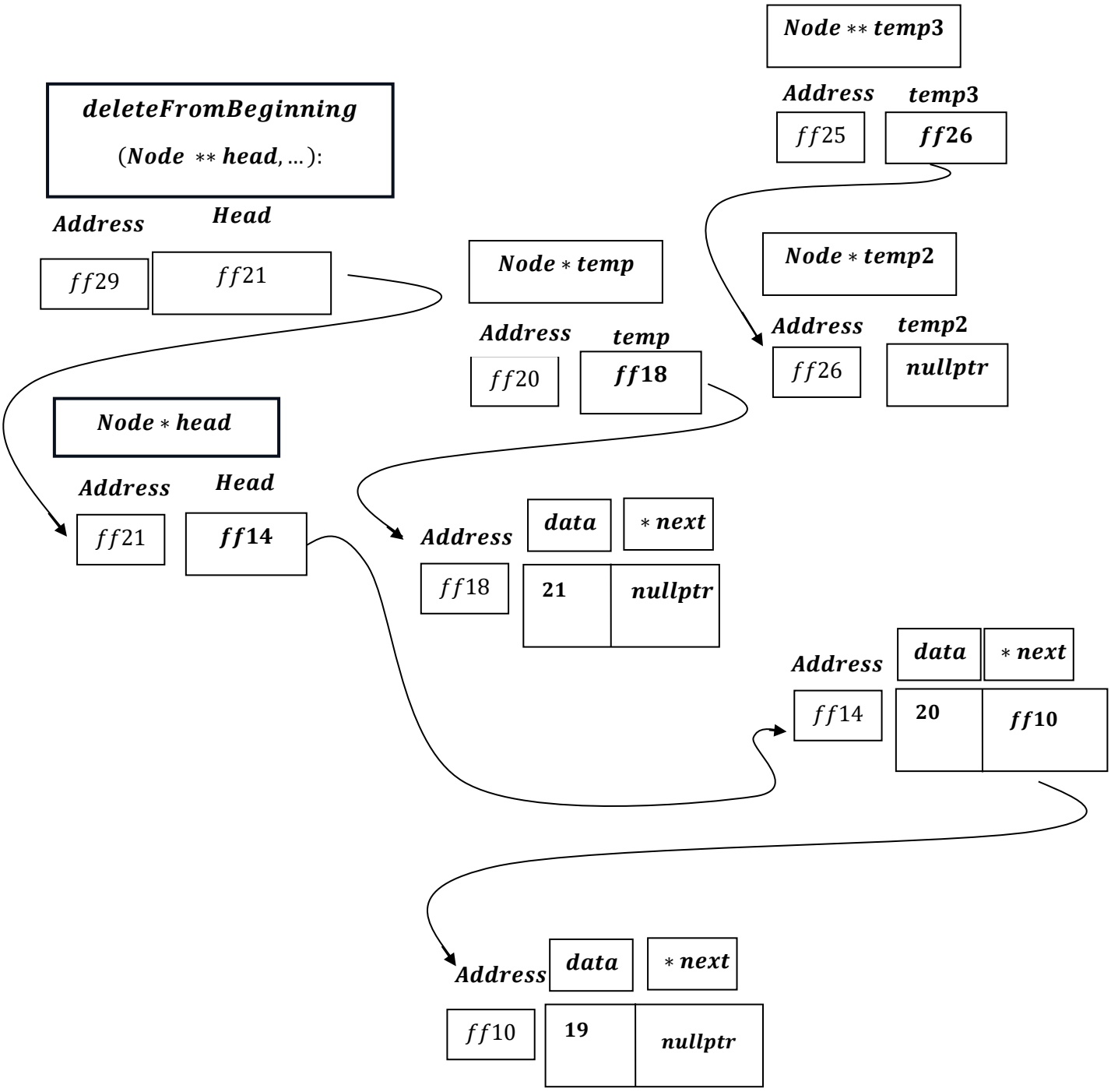
And now,

```
temp->next = nullptr;
```



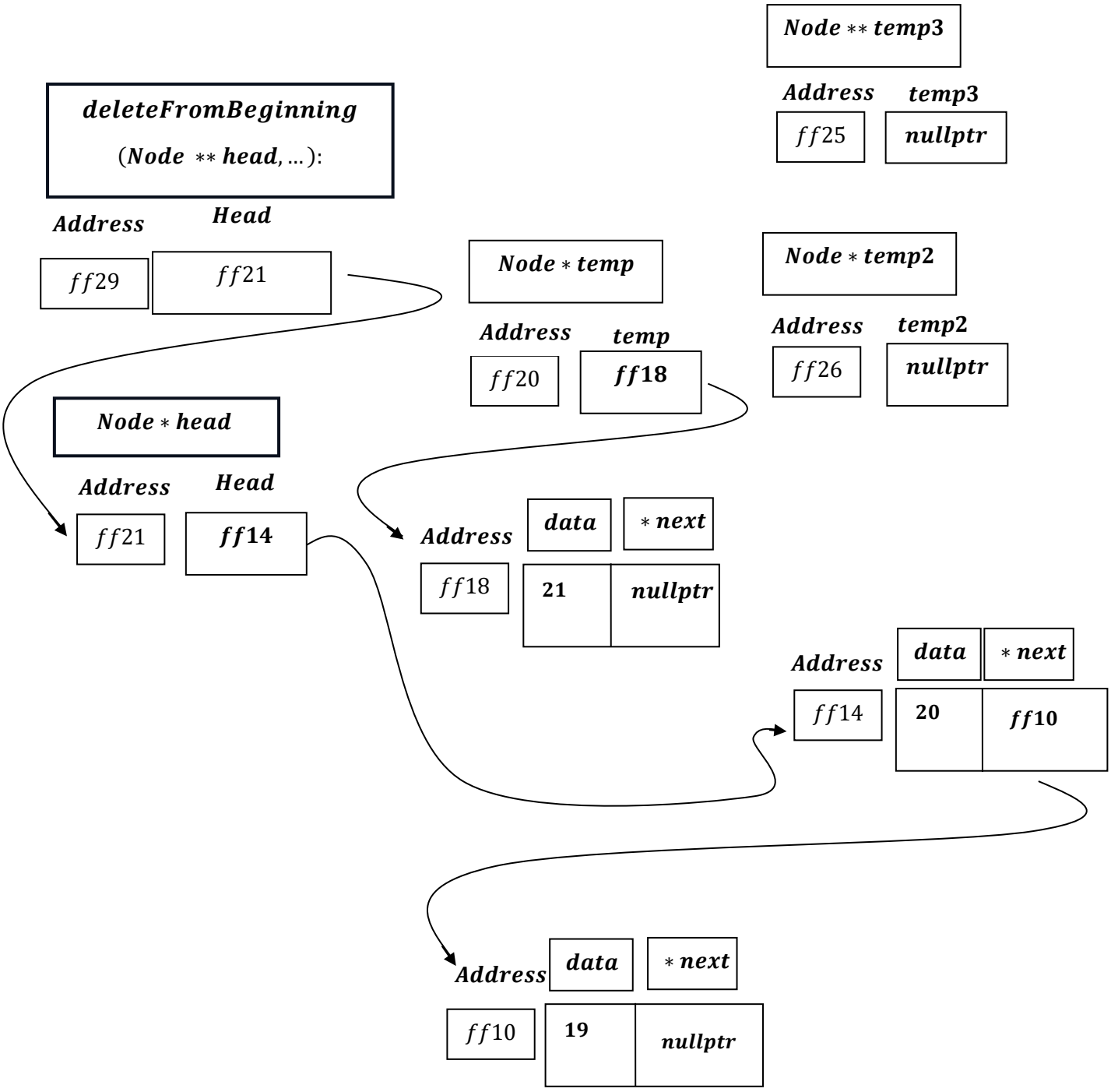
Next:

```
temp2 = nullptr;
```



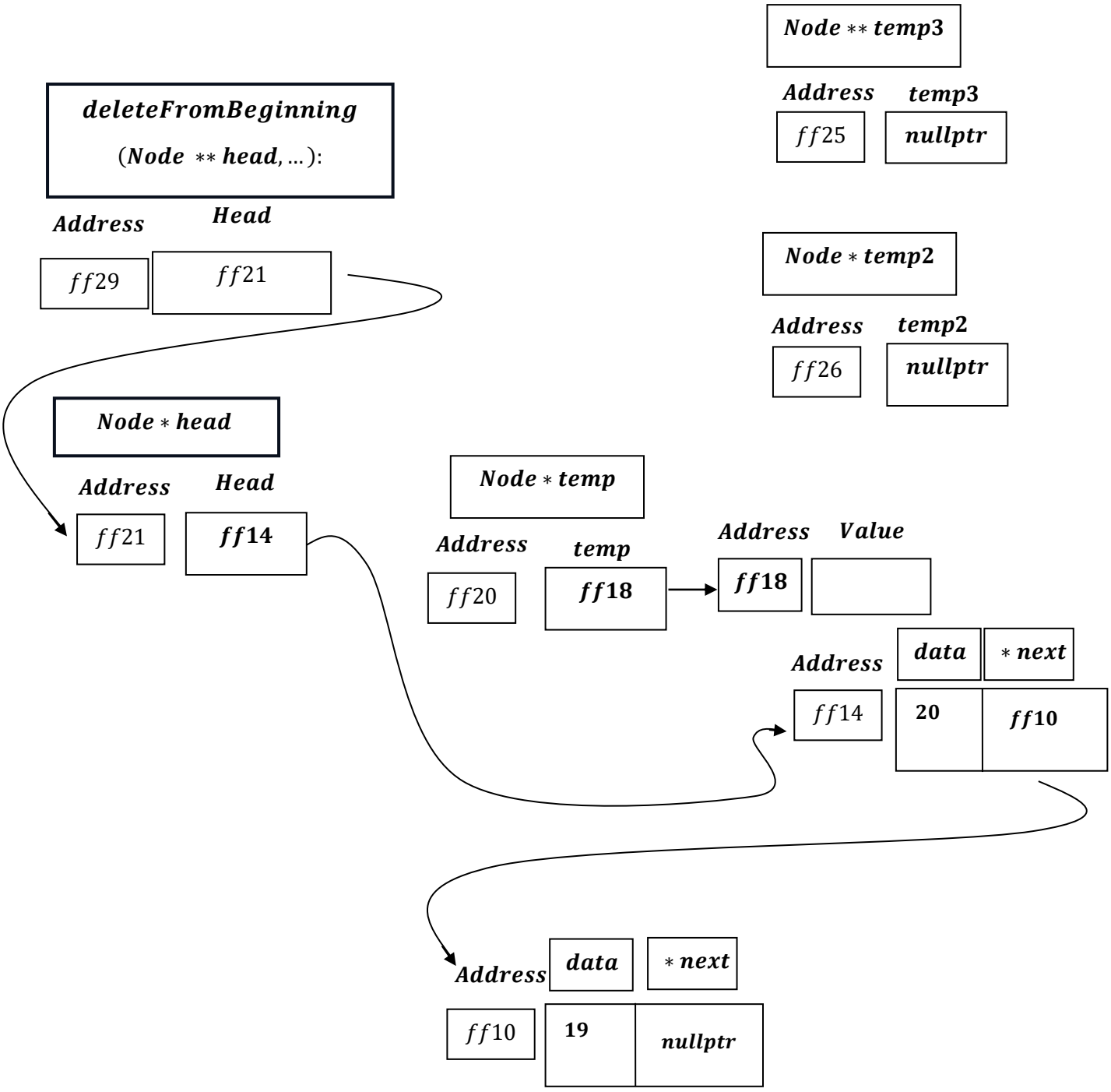

```
    temp3 = nullptr;
```

Note , this is not necessary just to show how change Node ** temp3's value (here it is assigned to nullptr).



Now,

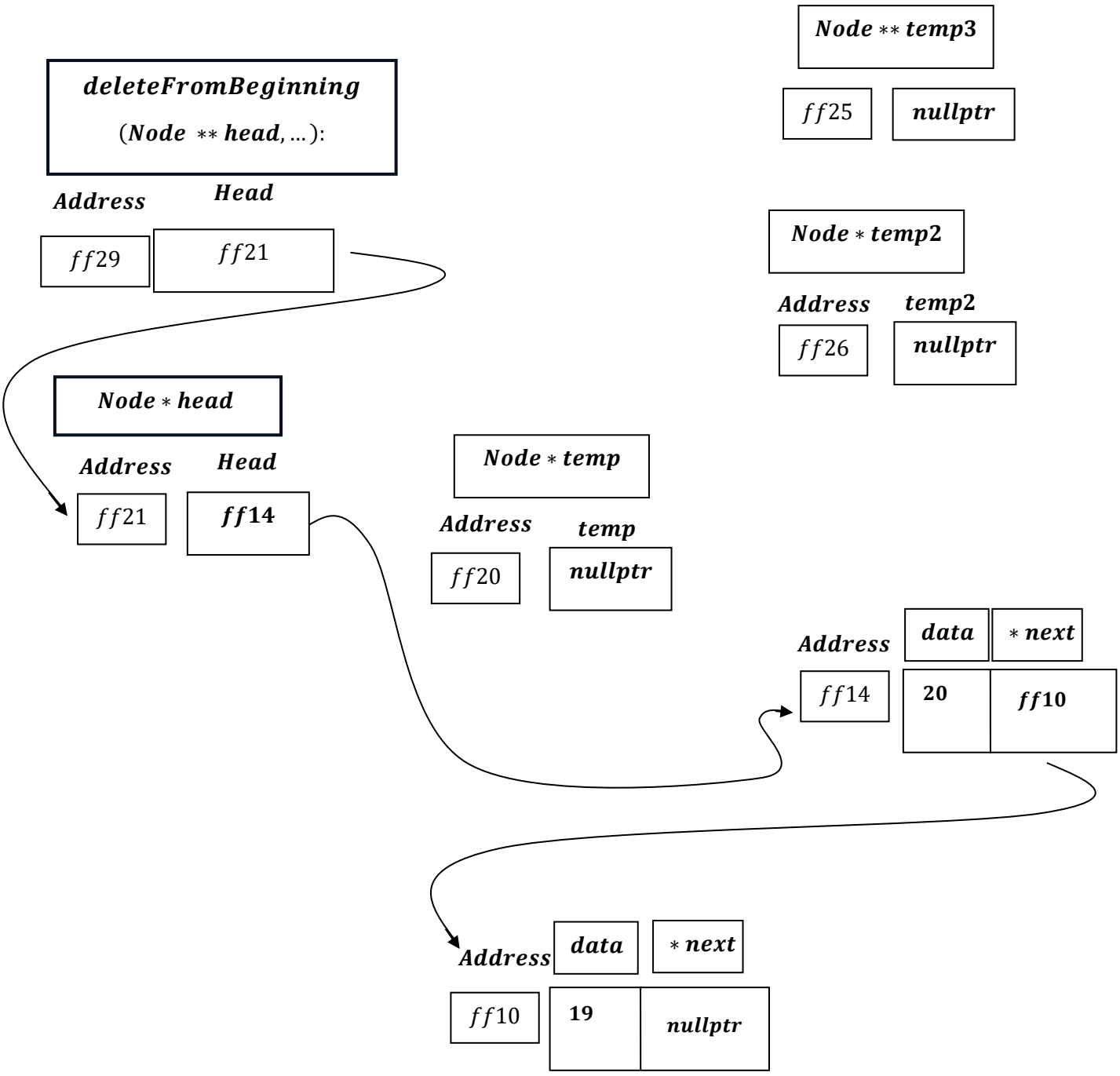
```
free(temp);
```



After freeing the memory location , temp variable is still present pointing invalid memory location i.e. now temp is a dangling pointer.

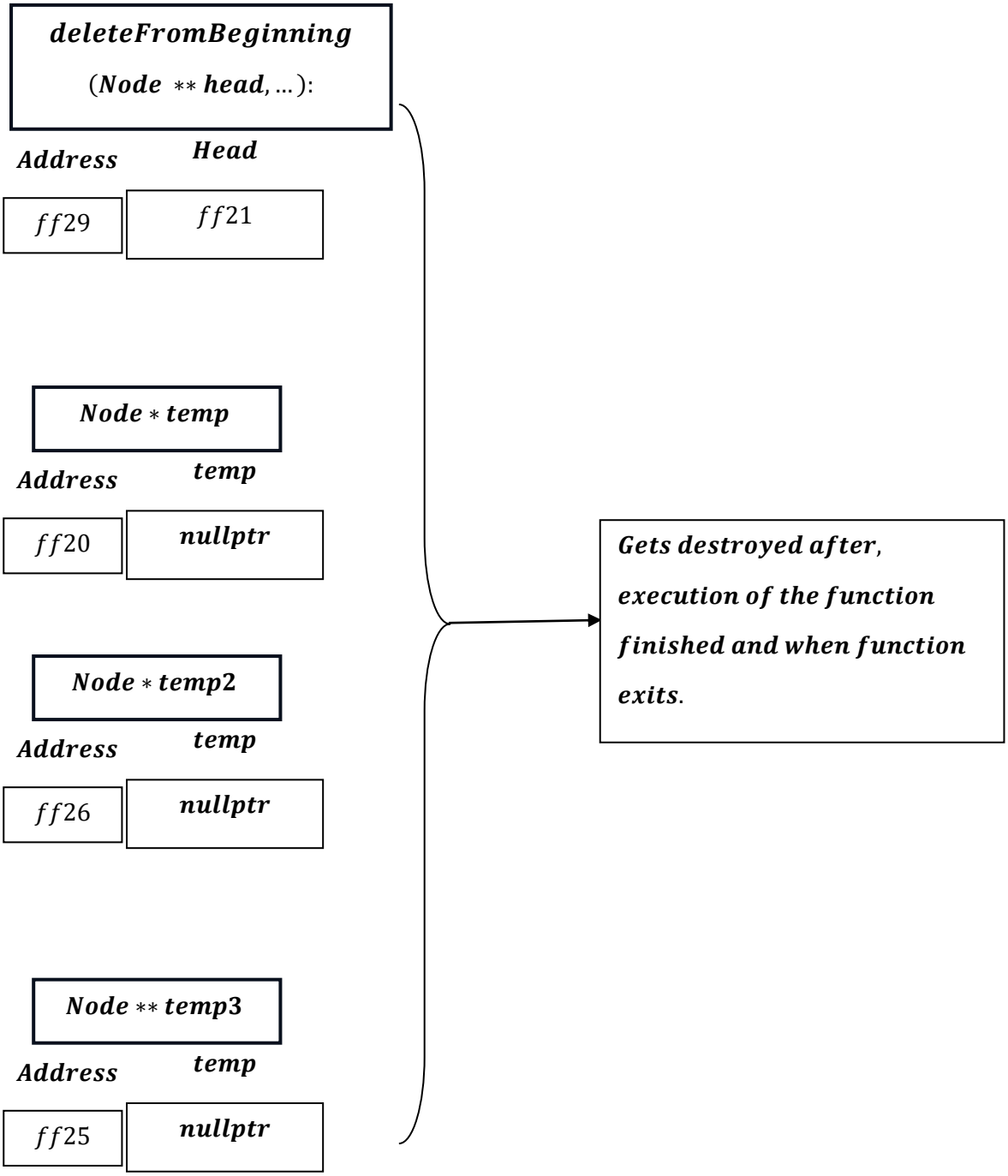
Hence,

```
temp = nullptr;
```

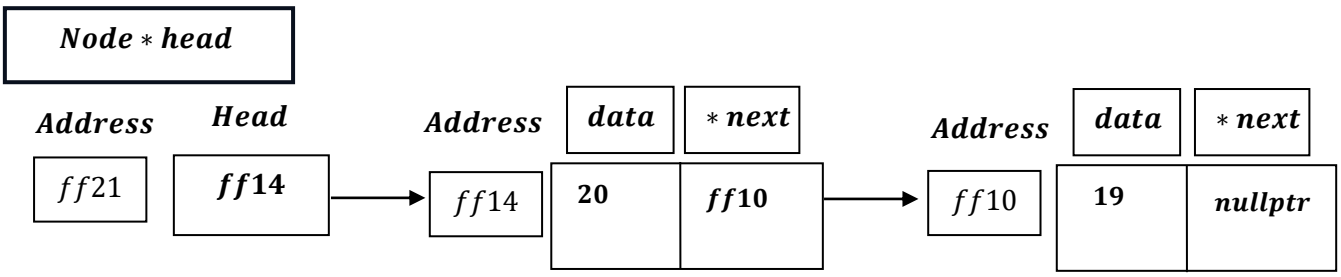


After the function module ends , all temporary pointer variables created gets destroyed.

i.e.



And we are left with:



Example 2 coded as:

```
void deleteFromBeginning(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *temp = *head;
        Node *temp2 = *head;
        Node **temp3= &temp2;
        *temp3 = (*temp3)->next;
        *head = *temp3;
        cout << "Deleted node: " << temp->data << endl;
        temp->next = nullptr;
        temp2 = nullptr;
        temp3 = nullptr;
        free(temp);
        temp = nullptr;
    }
    cout << "Node is deleted successfully." << endl;
}
```

Now with both Example 1 and Example 2 presentation , both the example is just for understanding , and these examples increases lines of code , increases space memory because of unnecessary extra variables , hence the first method we have used is more optimised than these examples.

Hence considering the first method:

```
void deleteFromBeginning(Node **head)

{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *temp = *head;
        *head = (*head)->next;
        cout << "Deleted node: " << temp->data << endl;
        temp->next = nullptr;
        free(temp);
        temp = nullptr;
    }
    cout << "Node is deleted successfully." << endl;
}
```

The deleteFromBeginning function step by step:

1. *Check if the List is Empty.*

2. *If the list is not empty, it stores the current head node in a temporary pointer `temp`.*

3. *The head pointer is updated to point to the next node in the list. This effectively removes the first node from the list.*

4. *The function then prints the data of the node that is being deleted.*

5. *The `next` pointer of the deleted node is set to `nullptr` to avoid any potential dangling pointers. The `next` pointer earlier pointed to the memory location at which head pointer is pointing to the current*

head node. The `next` node of the deleted node is set to nullptr so that the deleted node stops pointing to the current head node. But if it remains pointing then it becomes a potential dangling pointer.

6. *The memory allocated for the deleted node is freed using `free()`.*

7. *The temporary pointer `temp` is set to `nullptr` to avoid any dangling pointers.*

8. *Finally, the function prints a success message indicating that the node has been deleted successfully.*

9. *This ensures that the list remains consistent and that there are no memory leaks or dangling pointers.*

Time Complexity Of Delete From Beginning

```
void deleteFromBeginning(Node **head)

{
    if (*head == nullptr)          → O(1)
    {
        cout << "Error: List is empty." << endl;   → O(1)
        return;   → O(1)
    }                                              → O(1)
    else
    {
        Node *temp = *head;   → O(1)
        *head = (*head)->next;   → O(1)
        cout << "Deleted node: " << temp->data << endl;   → O(1)
        temp->next = nullptr;   → O(1)
        free(temp);   → O(1)
        temp = nullptr;   → O(1)
    }
    cout << "Node is deleted successfully." << endl;   → O(1)
}
```

1. ***Checking if the list is empty***:

$$if\ (*\ head\ ==\ nullptr)$$

- ***This is a simple pointer comparison.***
- ***Time Complexity***: $O(1)$ ***(constant time)***
- ***Explanation***: ***Regardless of the size of the list, this operation always takes the same.***

2. ***If the list is empty***:

$$cout\ <<\ "Error: List\ is\ empty."\ <<\ endl;$$

$$return;$$

$-$ *Printing an error message and returning.*

$-$ *Time Complexity*: $O(1)$(*constant time*).

$-$ *Explanation*: *Printing a fixed string and returning are both constant time operations.*

3. *If the list is not empty*:

$Node * temp = * head;$

$-$ *Assigning the head pointer to a temporary variable.*

$-$ *Time Complexity*: $O(1)$(*constant time*).

$-$ *Explanation*: *This is a simple pointer assignment, which always takes the same amount of time.*

4. *Updating the head pointer*:

$* head = (* head) -> next;$

$-$ *Moving the head pointer to the next node.*

$-$ *Time Complexity*: $O(1)$ (*constant time*)

$-$ *Explanation*: *This is another pointer assignment, which is a constant time operation.*

5. *Printing the deleted node's data*:

$cout << "Deleted\ node:" << temp -> data << endl;$

$-$ *Outputting the data of the node being deleted.*

$-$ *Time Complexity*: $O(1)$ (*constant time*)

$-$ *Explanation*: *Printing a single value is a constant time operation, regardless of the size of the list.*

6. *Clearing the next pointer of the deleted node*:

$temp -> next = nullptr;$

$-$ *Setting the next pointer of the deleted node to null.*

$-$ *Time Complexity*: $O(1)$(*constant time*).

$-$ $Explanation$: $This\ is\ a\ simple\ pointer\ assignment,$

$which\ is\ always\ constant\ time.$

7. $Freeing\ the\ memory$:

$free(temp);$

$-$ $Deallocating\ the\ memory\ of\ the\ deleted\ node.$

$-$ $Time\ Complexity$: $O(1)$ $(constant\ time)$

$-$ $Explanation$: $Freeing\ a\ single\ node\ is\ a\ constant\ time\ operation$

$in\ most\ memory\ management\ systems.$

8. $Nullifying\ the\ temporary\ pointer$:

$temp\ =\ nullptr;$

$-$ $Setting\ the\ temporary\ pointer\ to\ null.$

$-$ $Time\ Complexity$: $O(1)$ $(constant\ time)$

$-$ $Explanation$: $This\ is\ a\ simple\ pointer\ assignment,$

$which\ is\ always\ constant\ time.$

9. $Printing\ success\ message$:

$cout\ <<\ "Node\ is\ deleted\ successfully."\ <<\ endl;$

$-$ $Outputting\ a\ success\ message.$

$-$ $Time\ Complexity$: $O(1)$ $(constant\ time)$

$-$ $Explanation$: $Printing\ a\ fixed\ string\ is\ a\ constant$

$time\ operation.$

<u>$Overall\ Time\ Complexity$</u>:

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1).$$

$-$ $Every\ operation\ in\ this\ function\ is\ O(1).$

$-$ $When\ we\ combine\ constant\ time\ operations, the\ result\ is\ still\ constant\ time.$

$-$ *Therefore, the overall time complexity of `deleteFromBeginning` is* $O(1)$.

Conclusion:

The `deleteFromBeginning` function has a time complexity of $O(1)$, *which means it performs in constant time regardless of the size of the linked list. This is because it only operates on the head of the list and does not need to traverse the entire list.*

Note, by relation: $\Omega \leq \Theta \leq O$, *we get*:

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq 1 \leq c_2 \times 1$

Singly Linked List	*Cases*	*Time Complexity*
Deletion From Beginning	*Worst Case* = *Best Case* [*Hence no true Average Case*[*As we delete only from first and hence no random selection*]]. *Hence, we can say,* *Worst Case* = *Best Case* = *Average Case*.	$\Omega(1) = \Theta(1) = O(1)$.

$1. Best\ Case: The\ list\ is\ empty. The\ function\ checks\ if\ the\ head\ is\ `nullptr`\ and\ returns\ immediately. This\ is\ an\ O(1)\ operation.$

$2. Worst\ Case: The\ list\ is\ not\ empty. The\ function\ deletes\ the\ first\ node\ and\ updates\ the\ head\ pointer. This\ involves\ a\ few\ pointer\ manipulations\ and\ is\ also\ an\ O(1)\ operation.$

$Since\ the\ function's\ behavior\ does\ not\ depend\ on\ the\ size\ of\ the\ list\ and\ always\ performs\ a\ constant\ amount\ of\ work, there\ is\ no\ variation\ in\ its\ time\ complexity. Therefore, there\ is\ no\ distinct\ average\ case;\ the\ time\ complexity\ remains\ O(1)\ in\ all\ scenarios.$

****